WILP, BITS Pilani

DEEP NEURAL NETWORKS \TEXTPERIODCENTERED\ COMPANION READER

From Perceptrons to Convolutional Architectures

A complete guide to artificial neurons, linear models, multi-class classification, feed-forward networks, and CNN building blocks.

Session 8 \textperiodcentered\ Read alongside the lecture slides \textperiodcentered\ Every slide example worked out in full

\smallskip \noindent{\small\color{mutedtext}% Prepared for the Work Integrated Learning Programmes (WILP), BITS Pilani.} \smallskip \noindent**How to use this companion.** The slides give you formulas and architecture diagrams. This companion explains the *why* behind every equation and step. We build every model from simple weighted sums up to deep convolutional neural networks (CNNs). Colour-coded callout boxes guide your reading: \textcolor{intuFrame}{**blue**} for intuition, \textcolor{everyFrame}{**amber**} for real-life pictures, \textcolor{workFrame}{**green**} for fully worked numerical examples, \textcolor{watchFrame}{**red**} for common pitfalls, and \textcolor{keyFrame}{**purple**} for core takeaways. \medskip

First taste: The artificial neuron and the perceptron

Multiply features by weights, add a bias, and pass the sum through a threshold function.

An **artificial neuron** is the atomic building block of deep learning. It takes an input feature vector $\mathbf{x} = [x_1, x_2, \ldots, x_d]^T$, multiplies each feature by a weight w_i, adds a bias term b, and computes a weighted score z:

$$z = \sum_{i=1}^{d} w_i x_i + b = \mathbf{w}^T \mathbf{x} + b.$$

The neuron then applies an **activation function** $f(z)$ to produce the final output $y = f(z)$. The simplest form of an artificial neuron is the **perceptron**. It uses a step function as its activation function:

$$f(z) = \{1 \quad \text{if } z \geq 00 \quad \text{if } z < 0$$

☞ The intuition

Think of a neuron as a committee member scoring a proposal. Each input x_i is an evidence item. The weight w_i is how much the member trusts that item. The bias b is the baseline willingness to say yes. If the weighted total meets the threshold, the output fires.

★ Everyday picture

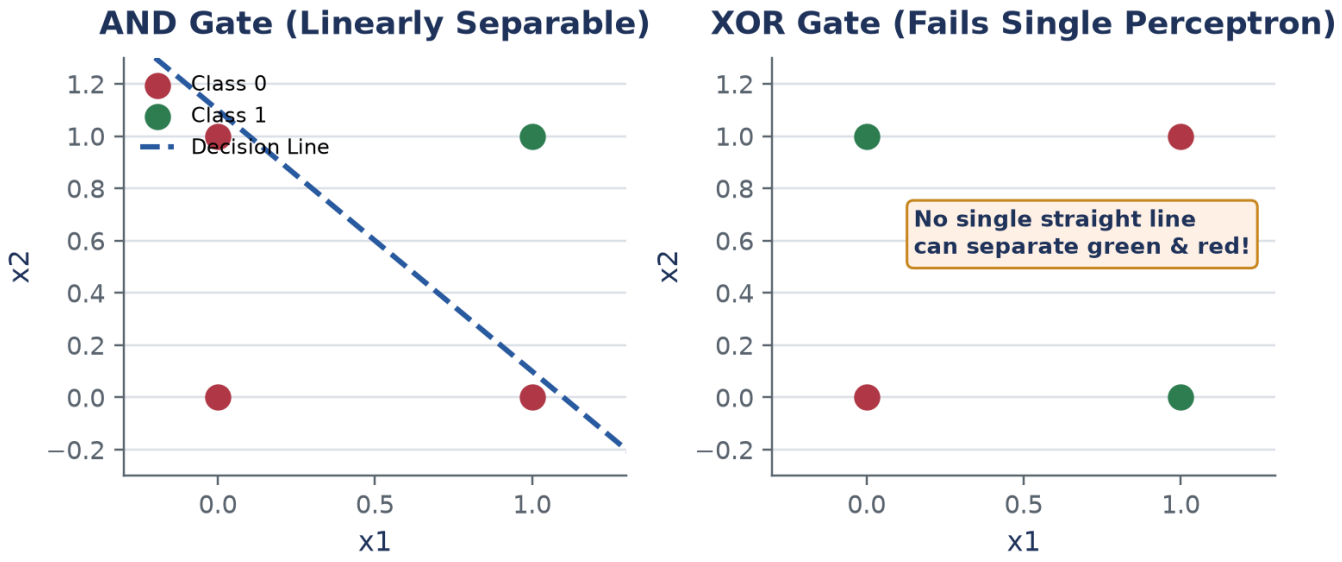
Deciding whether to attend an outdoor concert. Inputs: $x_1 = 1$ (good weather), $x_2 = 1$ (favourite band). If weather matters more ($w_1 = 3$) than the band ($w_2 = 2$) and your threshold is $b = -4$, you attend only if $3x_1 + 2x_2 - 4 \geq 0$.

A single perceptron can learn simple logical gates like **AND** or **OR**. For an AND gate with two inputs $x_1, x_2 \in \{0, 1\}$, set weights $w_1 = 1, w_2 = 1$ and bias $b = -1.5$. The weighted score is $z = 1 \cdot x_1 + 1 \cdot x_2 - 1.5$. When both inputs are 1, $z = 0.5 \geq 0 \implies y = 1$. Otherwise, $z < 0 \implies y = 0$. However, a single perceptron **fails on the XOR gate**. An XOR gate outputs 1 when $x_1 \neq x_2$ and 0 when $x_1 = x_2$. Plotting the four input points $(0,0), (0,1), (1,0), (1,1)$ on a plane shows that no single straight line can separate the two classes. A single perceptron can only draw a linear decision boundary. To solve non-linearly separable problems like XOR, we must stack neurons into multi-layer networks.

✎ Worked example: Why a single perceptron cannot solve XOR

Consider the XOR truth table points: $(0,0) \to 0$, $(1,1) \to 0$, $(0,1) \to 1$, $(1,0) \to 1$.

1. **Write linear inequalities for output 0.** For $(0,0) \to 0$, we need $w_1(0) + w_2(0) + b < 0 \implies b < 0$. For $(1,1) \to 0$, we need $w_1 + w_2 + b < 0$.
2. **Write linear inequalities for output 1.** For $(1,0) \to 1$, we need $w_1 + b \geq 0$. For $(0,1) \to 1$, we need $w_2 + b \geq 0$.
3. **Sum the inequalities for output 1.** $(w_1 + b) + (w_2 + b) \geq 0 \implies w_1 + w_2 + 2b \geq 0$.
4. **Derive the contradiction.** Since $b < 0$, we have $w_1 + w_2 + b > w_1 + w_2 + 2b \geq 0 \implies w_1 + w_2 + b > 0$. This directly contradicts $w_1 + w_2 + b < 0$ required for $(1,1)$. No weights (w_1, w_2, b) exist.



Linear boundary for AND gate vs non-linear boundary requirement for XOR gate.

✗ Watch out

Do not confuse single perceptrons with multi-layer feed-forward networks. A single perceptron has no hidden layers and can only draw flat, linear decision lines.

✓ Key takeaway

A single perceptron computes a linear decision boundary. It solves linearly separable logic like AND, but fails on non-linear patterns like XOR. Solving complex problems requires stacking hidden layers.

Predicting numbers: Linear regression and optimization

Predict a continuous output as a weighted sum of inputs, and adjust weights using gradient descent.

When our target $y \in \mathbb{R}$ is continuous (like house prices or temperature), we use **linear regression**. The mathematical model is:

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d = \mathbf{w}^T \mathbf{x} + b$$

Linear regression can be viewed as a single artificial neuron with an **identity activation function** $f(z) = z$. The identity function has key properties: \begin{enumerate} [leftmargin=1.5em]
Continuous output over $(-\infty, \infty)$.
Differentiable everywhere with derivative $f'(z) = 1$.
Allows gradients to flow backward unchanged during optimization. \end{enumerate} To measure how well our model fits the training data, we define an objective function called **squared error loss**:

$$L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$$

For a dataset of N examples, the Mean Squared Error (MSE) cost function is $J(\mathbf{w}, b) = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{2}(y^{(i)} - \hat{y}^{(i)})^2$. To minimize this loss, we compute the gradient with respect to weights and update them using Stochastic Gradient Descent (SGD):

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_j} = -(y - \hat{y}) \cdot 1 \cdot x_j = (\hat{y} - y) x_j w_j^{(\text{new})} \qquad = w_j^{(\text{old})} - \eta \frac{\partial L}{\partial w_j}$$

where $\eta > 0$ is the learning rate. To evaluate model performance, we use the R^2 **score** (Coefficient of Determination):

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

An $R^2 = 1$ indicates perfect prediction, while $R^2 = 0$ means the model performs no better than predicting the mean $\bar{y}$.

✎ Worked example: One SGD update step for Linear Regression

Let a training sample be $\mathbf{x} = [2.0, 1.0]^T$ with true target $y = 5.0$. Initial weights are $\mathbf{w} = [0.5, -0.5]^T$, bias $b = 0.1$, and learning rate $\eta = 0.1$.

1. **Compute prediction.** $\hat{y} = w_1 x_1 + w_2 x_2 + b = (0.5)(2.0) + (-0.5)(1.0) + 0.1 = 1.0 - 0.5 + 0.1 = 0.6$.
2. **Compute initial loss.** $L = \frac{1}{2}(y - \hat{y})^2 = \frac{1}{2}(5.0 - 0.6)^2 = \frac{1}{2}(4.4)^2 = 9.68$.
3. **Compute prediction error and gradients.** Error $(\hat{y} - y) = 0.6 - 5.0 = -4.4$.
   $\frac{\partial L}{\partial w_1} = (\hat{y} - y)x_1 = (-4.4)(2.0) = -8.8$.
   $\frac{\partial L}{\partial w_2} = (\hat{y} - y)x_2 = (-4.4)(1.0) = -4.4$.
   $\frac{\partial L}{\partial b} = (\hat{y} - y) = -4.4$.
4. **Update weights and bias.** $w_1^{(\text{new})} = 0.5 - 0.1(-8.8) = 0.5 + 0.88 = 1.38$.
   $w_2^{(\text{new})} = -0.5 - 0.1(-4.4) = -0.5 + 0.44 = -0.06$.
   $b^{(\text{new})} = 0.1 - 0.1(-4.4) = 0.1 + 0.44 = 0.54$.
5. **Compute new prediction and loss.** $\hat{y}_{\text{new}} = (1.38)(2.0) + (-0.06)(1.0) + 0.54 = 2.76 - 0.06 + 0.54 = 3.24$.
   $L_{\text{new}} = \frac{1}{2}(5.0 - 3.24)^2 = \frac{1}{2}(1.76)^2 = 1.5488$. Loss decreased from 9.68 to 1.55 in a single step!

✓ Key takeaway

Linear regression predicts continuous targets using identity activation. We optimize weights by moving in the opposite direction of the loss gradient, reducing prediction error.

Predicting categories: Logistic regression for binary classification

Squash weighted scores into probabilities using the sigmoid activation function.

In **classification**, our goal is to assign inputs to discrete categories. For binary classification, the target is $y \in \{0, 1\}$. Linear regression fails for classification because its outputs are unbounded $(-\infty, \infty)$ and sensitive to outliers. **Logistic regression** solves this by passing the score $z = \mathbf{w}^T\mathbf{x} + b$ through the **sigmoid activation function** $\sigma(z)$:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x} + b)}}$$

The sigmoid function maps any real number $z \in (-\infty, \infty)$ to a valid probability $\hat{y} \in (0, 1)$. Properties of Sigmoid: \begin{enumerate}[leftmargin=1.5em]
Output ranges between 0 and 1.
Smooth and differentiable everywhere, with derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
Thresholding: if $\hat{y} \geq 0.5 \implies$ Class 1, else Class 0. \end{enumerate} To train logistic regression, we use the **Binary Cross-Entropy (BCE) loss**:

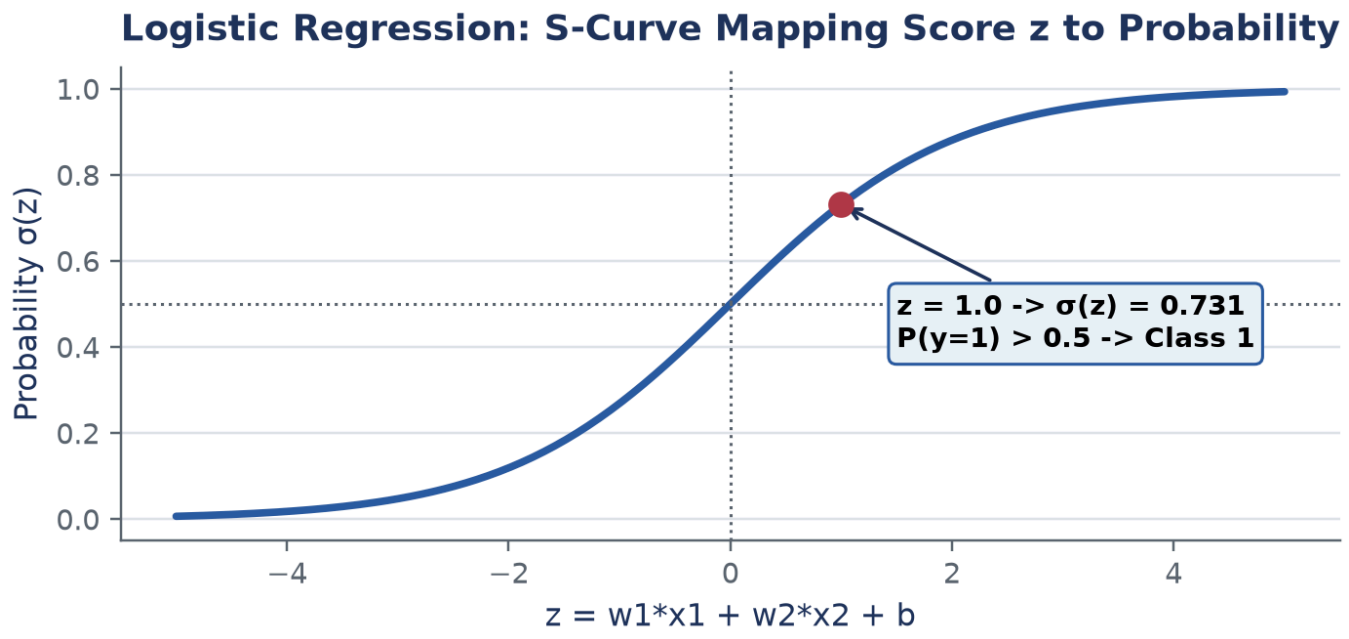
$$L(y, \hat{y}) = -\left[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\right]$$

When true label $y = 1$, loss is $-\log(\hat{y})$. If prediction $\hat{y} \to 1$, loss $\to 0$. If $\hat{y} \to 0$, loss $\to \infty$. Differentiating BCE loss with respect to z yields a remarkably clean gradient:

$$\frac{\partial L}{\partial z} = \hat{y} - y$$

Hence, weight update for feature x_j is simply:

$$w_j^{(\text{new})} = w_j - \eta(\hat{y} - y)x_j$$



Sigmoid activation function mapping unbounded scores z into probability range (0, 1).

To evaluate classification performance beyond simple accuracy, we use a **confusion matrix**: \begin{center} \small \begin{tabular}{@{}ccc@{}} \toprule & **Predicted Positive (1)** & **Predicted Negative (0)**
\midrule **Actual Positive (1)** & True Positive (TP) & False Negative (FN)
Actual Negative (0) & False Positive (FP) & True Negative (TN)
\bottomrule \end{tabular} \end{center} Standard classification metrics are:

$$\textbf{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \textbf{Precision} \quad = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad \textbf{(Of all predicted positives, how many were right?)} \textbf{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \textbf{(Of all act}$$

✎ Worked example: Two SGD steps for Logistic Regression

Given input $\mathbf{x} = [1.0, 2.0]^T$, true label $y = 1$, initial weights $\mathbf{w} = [0.1, -0.2]^T$, bias $b = 0.0$, and learning rate $\eta = 0.5$.

1. **Iteration 1: Score and prediction.**
   $z^{(1)} = (0.1)(1.0) + (-0.2)(2.0) + 0.0 = 0.1 - 0.4 = -0.3$.
   $\hat{y}^{(1)} = \sigma(-0.3) = \frac{1}{1+e^{0.3}} \approx \frac{1}{1+1.34986} \approx 0.4255$.
2. **Iteration 1: Error and update.**
   Error $(\hat{y} - y) = 0.4255 - 1.0 = -0.5745$.
   $w_1^{(1)} = 0.1 - 0.5(-0.5745)(1.0) = 0.1 + 0.28725 = 0.38725$.
   $w_2^{(1)} = -0.2 - 0.5(-0.5745)(2.0) = -0.2 + 0.5745 = 0.3745$.
   $b^{(1)} = 0.0 - 0.5(-0.5745) = 0.28725$.
3. **Iteration 2: Score and prediction with new weights.**
   $z^{(2)} = (0.38725)(1.0) + (0.3745)(2.0) + 0.28725 = 0.38725 + 0.7490 + 0.28725 = 1.4235$.
   $\hat{y}^{(2)} = \sigma(1.4235) = \frac{1}{1+e^{-1.4235}} \approx \frac{1}{1+0.24087} \approx 0.8059$.
4. **Check progress.**
   Predicted probability for class 1 increased from 0.4255 to 0.8059. The model is rapidly learning!

✓ Key takeaway

Logistic regression uses sigmoid activation to output probabilities for binary classification. Binary cross-entropy loss penalizes confident wrong predictions.

Multi-class classification and the Softmax function

Convert raw output scores for K classes into a normalized probability distribution.

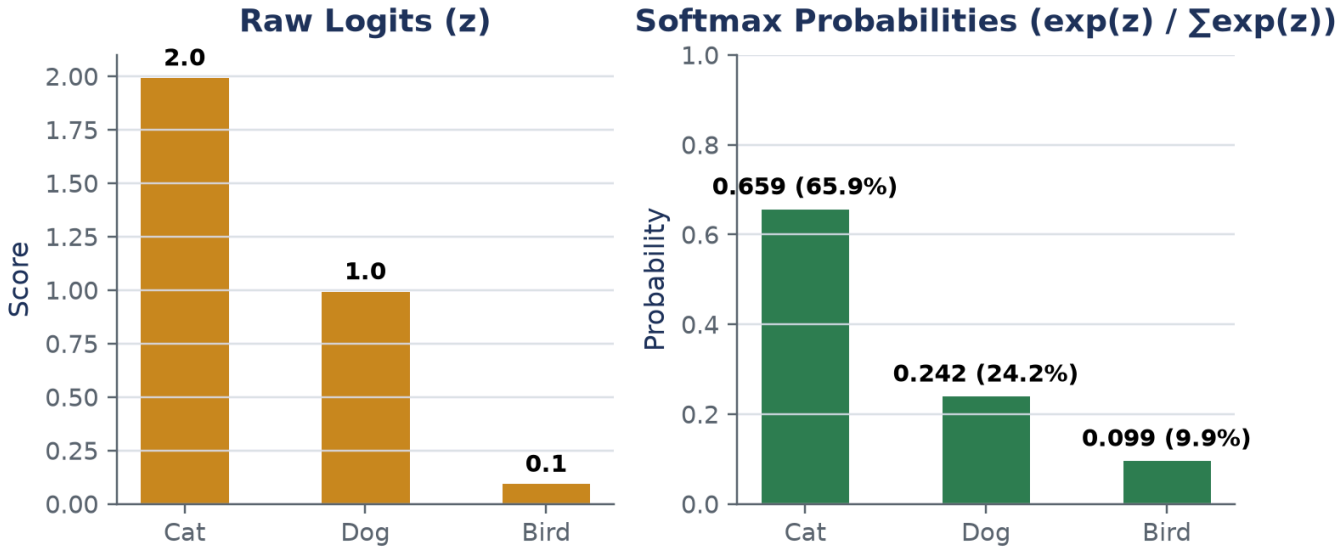
When a classification task has $K > 2$ discrete classes, we format target labels using **one-hot encoding**. For class k, label vector $\mathbf{y}$ has length K with 1 at position k and 0 elsewhere. For example, with $K = 3$ classes: Class 1 is $[1, 0, 0]^T$, Class 2 is $[0, 1, 0]^T$, Class 3 is $[0, 0, 1]^T$. For K classes, the network output layer produces K raw real-valued scores called **logits** $\mathbf{z} = [z_1, z_2, \ldots, z_K]^T$. We convert logits into class probabilities $P(y = k \mid \mathbf{x})$ using the **Softmax function**:

$$\hat{y}_k = \textbf{Softmax}(z_k) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

Properties of Softmax: \begin{enumerate}[leftmargin=1.5em]
Every output $\hat{y}_k > 0$.
The sum of all class probabilities equals 1: $\sum_{k=1}^{K} \hat{y}_k = 1$.
Exponentials amplify differences between top logits, making the dominant class stand out. \end{enumerate} The loss function for multi-class classification is **Categorical Cross-Entropy Loss**:

$$L(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k = -\log \hat{y}_c$$

where c is the true class label.



Softmax operation converting raw logits z into a sum-to-one probability distribution.

✎ Worked example: Softmax computation and cross-entropy loss

Given raw logits $\mathbf{z} = [2.0, 1.0, 0.1]^T$ for a 3-class problem where true class is Class 1 ($y = [1, 0, 0]^T$).

1. **Compute exponentials.**
   $e^{z_1} = e^{2.0} \approx 7.3891$.
   $e^{z_2} = e^{1.0} \approx 2.7183$.
   $e^{z_3} = e^{0.1} \approx 1.1052$.
2. **Compute sum of exponentials.**
   $S = 7.3891 + 2.7183 + 1.1052 = 11.2126$.
3. **Compute Softmax probabilities.**
   $\hat{y}_1 = 7.3891 / 11.2126 \approx 0.6590 \quad (65.9\%)$.
   $\hat{y}_2 = 2.7183 / 11.2126 \approx 0.2424 \quad (24.2\%)$.
   $\hat{y}_3 = 1.1052 / 11.2126 \approx 0.0986 \quad (9.9\%)$.
   Check sum: $0.6590 + 0.2424 + 0.0986 = 1.0000$.
4. **Compute Categorical Cross-Entropy Loss.**
   $L = -\log(\hat{y}_1) = -\log(0.6590) \approx 0.4170$.

✓ Key takeaway

Softmax converts unbounded logits into a normalized probability distribution across multiple classes. Combined with cross-entropy loss, it drives model training for multi-class classification.

Deep Feed-Forward Neural Networks (DFNN)

Stack hidden layers of non-linear transformations to build expressive deep representations.

A **Deep Feed-Forward Neural Network (DFNN)** consists of an input layer, one or more hidden layers, and an output layer. Information moves strictly forward through the network. For layer $l \in \{1, 2, \ldots, L\}$:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\mathbf{a}^{(l)} \qquad = f^{(l)}(\mathbf{z}^{(l)})$$

where $\mathbf{a}^{(0)} = \mathbf{x}$ is the input feature vector, $\mathbf{W}^{(l)}$ is the weight matrix of shape $(n_l \times n_{l-1})$, $\mathbf{b}^{(l)}$ is the bias vector of length n_l, and $f^{(l)}$ is a non-linear activation function (like ReLU or Sigmoid). Calculating total learnable parameters in a feed-forward network: For layer l connected to layer $l-1$:

$$\text{Params}^{(l)} = n_l \times n_{l-1} + n_l = n_l(n_{l-1} + 1)$$

Total network parameters equal the sum of parameters across all layers.

✎ Worked example: Forward propagation through a 2-layer network

Let input $\mathbf{x} = [1, 2]^T$.
Layer 1 (Hidden, 2 units, ReLU activation): $\mathbf{W}^{(1)} = [1 \quad 0 - 1 \quad 2]$, $\mathbf{b}^{(1)} = [01]$.
Layer 2 (Output, 1 unit, Linear activation): $\mathbf{W}^{(2)} = [2 \quad -1]$, $b^{(2)} = 0.5$.

1. **Hidden layer weighted sum.**
   $\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = [1(1) + 0(2) - 1(1) + 2(2)] + [01] = [13] + [01] = [14]$.
2. **Hidden layer activation (ReLU).**
   $\mathbf{a}^{(1)} = \mathbf{ReLU}([14]) = [14]$.
3. **Output layer weighted sum.**
   $z^{(2)} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + b^{(2)} = [2 \quad -1]\,[14] + 0.5 = (2 - 4) + 0.5 = -1.5$.
4. **Final network prediction.**
   Since output activation is identity, $\hat{y} = a^{(2)} = -1.5$.

\noindent**Why depth matters.** Deep networks learn **hierarchical feature representations**: early layers learn simple low-level primitives (edges, color blobs), middle layers combine primitives into shapes and parts, and deep layers assemble parts into complex high-level concepts (faces, cars). Theoretical results show that deep networks can represent complex functions exponentially more efficiently than shallow, wide networks with the same number of parameters. \noindent**Challenges in training deep networks:** \begin{enumerate}[leftmargin=1.5em]
Vanishing Gradients: Gradients shrink exponentially as they backpropagate through deep layers, causing early layers to stop updating. Solution: ReLU activations, residual skip connections, proper weight initialization.
Exploding Gradients: Gradients blow up exponentially, causing numerical instability (NaN values). Solution: Gradient clipping, normalization layers.
Degradation Problem: Beyond a certain depth, standard deep networks suffer higher training error. Solution: Residual connections (ResNets). \end{enumerate}

✓ Key takeaway

Deep networks compose simple linear transformations and non-linear activations to learn rich hierarchical representations. Residual connections enable training networks over 100 layers deep.

Convolutional Neural Networks (CNNs) and parameter efficiency

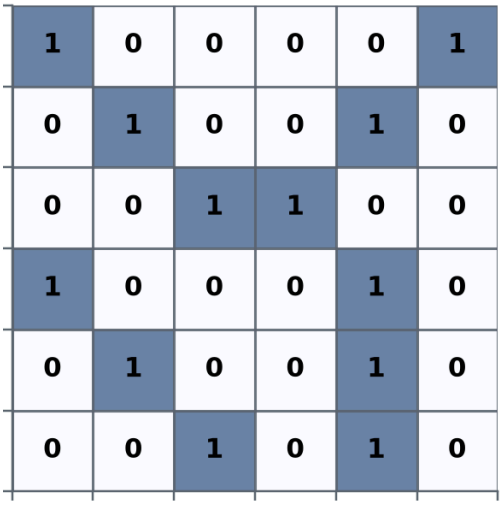
Preserve spatial structure using localized filters, parameter sharing, and spatial downsampling.

Standard fully connected networks fail on high-dimensional grid inputs like images (e.g., $224 \times 224 \times 3$ image flattened into a $150,528$-dimensional vector) because parameter counts explode, causing severe overfitting and losing 2D spatial relationships. **Convolutional Neural Networks (CNNs)** solve this using three spatial design principles: \begin{enumerate}[leftmargin=1.5em]
Local Receptive Fields: Filters look at small local patches of the input image at a time.
Parameter Sharing: The same filter weights slide across the entire input image.
Spatial Invariance: Features detected in one corner can be recognized anywhere in the image. \end{enumerate} \noindent**Convolution Output Spatial Dimensions Formula:** For an input grid of height/width N, kernel size K, padding P, and stride S:

$$\text{Output Shape } O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1$$

\noindent**Types of Convolution Operations:** \begin{itemize}[leftmargin=1.5em]
Standard Convolution: Sliding a $K \times K \times C_{\text{in}}$ filter across spatial dimensions.
Dilated (Atrous) Convolution: Inserts spaces (dilation rate r) between filter elements to expand receptive field without increasing parameter count. Effective kernel size becomes $K' = K + (K - 1)(r - 1)$.
Multi-Channel Convolution: Input has shape $(H \times W \times C_{\text{in}})$. Each of the C_{out} filters has shape $(K \times K \times C_{\text{in}})$. The filter convolves across all input channels simultaneously and sums the result to produce one 2D output feature map per filter. \end{itemize} \noindent**Pooling Operations (Spatial Downsampling):** Pooling reduces spatial dimensions (H, W) while preserving feature depth C. \begin{itemize}[leftmargin=1.5em]
Max Pooling: Extracts the maximum value in each window. Preserves prominent features and provides local translation invariance.
Average Pooling: Computes average value in each window. Smooths downsampling.
Crucial Property: Pooling layers have **ZERO learnable parameters**! \end{itemize}

Convolution Step: 6x6 Input Image -> 3x3 Filter -> 4x4 Output -> 2x2 Pool



Complete pipeline: 6x6 image convolved with 3x3 filter, passed through ReLU, and max-pooled.

✎ Worked example: Convolution, ReLU, and Max Pooling step-by-step

Given a 6×6 binary input matrix I, a 3×3 filter W with stride $S = 1, P = 0$, followed by ReLU and 2×2 Max Pooling with stride $S = 2$:

$$I = [1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 10 \quad 1 \quad 0 \quad 0 \quad 1 \quad 00 \quad 0 \quad 1 \quad 1 \quad 0 \quad 01 \quad 0 \quad 0 \quad 0 \quad 1 \quad 00 \quad 1 \quad 0 \quad 0 \quad 1 \quad 00 \quad 0 \quad 1 \quad 0 \quad 1 \quad 0], \quad W = [-1 \quad 1 \quad -1 - 1$$

1. **Calculate output shape after convolution.**
   $O = \frac{6-3}{1} + 1 = 4 \implies 4 \times 4$ feature map.
2. **Compute Convolution at top-left patch $I[0:3, 0:3]$.**
   Patch $= [1 \quad 0 \quad 00 \quad 1 \quad 00 \quad 0 \quad 1]$.
   Dot product: $(1)(-1) + (0)(1) + (0)(-1) + (0)(-1) + (1)(1) + (0)(-1) + (0)(-1) + (0)(1) + (1)(-1) = -1 + 1 - 1 = -1$.
3. **Convolved 4×4 Feature Map Result.**
   $Z = [-1 \quad -1 \quad -1 \quad -1 - 1 \quad -1 \quad -2 \quad -1 - 1 \quad -1 \quad -2 \quad 1 - 1 \quad 0 \quad -4 \quad 3]$.
4. **Apply ReLU Activation $\max(0, z)$.**
   $A = \text{ReLU}(Z) = [0 \quad 0 \quad 0 \quad 00 \quad 0 \quad 0 \quad 00 \quad 0 \quad 0 \quad 10 \quad 0 \quad 0 \quad 3]$.
5. **Apply 2×2 Max Pooling with Stride 2.**
   Output shape: $O_{\text{pool}} = \frac{4-2}{2} + 1 = 2 \implies 2 \times 2$.
   Top-left block $[0 \quad 00 \quad 0] \rightarrow \max = 0$.
   Bottom-right block $[0 \quad 10 \quad 3] \rightarrow \max = 3$.
   Final Pooled Output: $[0 \quad 00 \quad 3]$.

Parameter counting and CNN architecture design patterns

Master exact parameter calculations and architectural patterns for high-performance networks.

Understanding parameter counting is critical for designing efficient neural networks. \noindent**1. Convolutional Layer Parameter Formula:** For a Conv layer with kernel size $K \times K$, C_{in} input channels, C_{out} output filters, with bias:

$$\text{Params}_{\text{Conv}} = \left(K \times K \times C_{\text{in}} + 1\right) \times C_{\text{out}}$$

Note: Parameters depend ONLY on kernel dimensions and channels, **NOT** on spatial height or width! \noindent**2. Pooling Layer Parameter Formula:**

$$\text{Params}_{\text{Pool}} = 0$$

\noindent**3. Fully Connected (FC) Layer Parameter Formula:** For an input vector of length n connected to m output neurons:

$$\text{Params}_{\text{FC}} = m \times (n + 1) = m \cdot n + m$$

✎ Worked example: Full CNN Parameter Calculation

Consider a CNN with: Input: $32 \times 32 \times 3$ image.
Conv1: 64 filters of size 5×5, stride 1, padding 2.
Pool1: 2×2 max pool, stride 2.
Conv2: 128 filters of size 3×3, stride 1, padding 1.
FC1: Dense layer with 256 units.
Output FC: 10 units.

1. **Conv1 parameters and output shape.**
   $\text{Params}_{\text{Conv1}} = (5 \times 5 \times 3 + 1) \times 64 = (75 + 1) \times 64 = 76 \times 64 = 4,864$.
   Output shape: $\left(\frac{32+2(2)-5}{1} + 1\right) = 32 \implies 32 \times 32 \times 64$.
2. **Pool1 parameters and output shape.**
   $\text{Params}_{\text{Pool1}} = 0$.
   Output shape: $16 \times 16 \times 64$.
3. **Conv2 parameters and output shape.**
   $\text{Params}_{\text{Conv2}} = (3 \times 3 \times 64 + 1) \times 128 = (576 + 1) \times 128 = 577 \times 128 = 73,856$.
   Output shape: $16 \times 16 \times 128$.
4. **Flatten step before FC1.**
   Flatten $16 \times 16 \times 128 = 32,768$ input dimensions.
5. **FC1 parameters.**
   $\text{Params}_{\text{FC1}} = (32,768 + 1) \times 256 = 8,388,864$.
6. **Output FC parameters.**
   $\text{Params}_{\text{Output}} = (256 + 1) \times 10 = 2,570$.
7. **Total network parameters.**
   $4,864 + 0 + 73,856 + 8,388,864 + 2,570 = 8,470,154$ parameters (≈ 8.47 M).

\noindent**Key Architectural Design Patterns in Modern CNNs:** \begin{enumerate}[leftmargin=1.5em]
Spatial Reduction + Channel Expansion: As spatial dimensions (H, W) decrease via pooling, channel depth C increases (e.g., $32 \times 32 \times 64 \to 16 \times 16 \times 128 \to 8 \times 8 \times 256$). This maintains feature capacity.
Repetitive Building Blocks: Stacking standard modular blocks (like VGG 3x3 Conv blocks) simplifies network design.
Skip/Residual Connections (ResNet): Adds identity shortcut connections $y = F(\mathbf{x}) + \mathbf{x}$. Enables backpropagation through 100+ layers by letting gradients flow directly through identity paths.
Bottleneck Design: Uses 1×1 convolutions to shrink channel dimensions before expensive 3×3 convolutions, reducing parameters by up to $8\times$ without losing expressiveness.
Multi-Scale Processing (Inception): Applies $1 \times 1, 3 \times 3, 5 \times 5$ filters in parallel and concatenates outputs to capture features at multiple receptive field scales.
\end{enumerate}

✓ Key takeaway

Convolutional layers achieve immense parameter efficiency via weight sharing. Modern CNN architectures rely on spatial reduction, residual skip connections, and bottleneck designs to train deep models stably.

Glossary \& symbols

\addcontentsline{toc}{section}{Glossary \& symbols} \noindent**Key terms.** \begin{description}[leftmargin=9.5em,style=nextline,font=\normalfont\bfseries\color{navy}]
[Artificial Neuron] Basic computational unit performing weighted sum followed by non-linear activation.
[Perceptron] Simplest neuron with step function activation; computes linear decision boundaries.
[Sigmoid] S-shaped activation function mapping scores into range $(0, 1)$ for probabilities.
[Softmax] Function converting unnormalized logits into normalized multi-class probability distribution.
[Binary Cross-Entropy] Loss function measuring divergence between predicted probabilities and binary labels.
[Convolution Layer] Layer applying sliding learnable filters to preserve spatial feature structure.
[Max Pooling] Fixed operation extracting maximum value in spatial windows with zero parameters.
[Residual Connection] Identity shortcut connection $y = F(\mathbf{x}) + \mathbf{x}$ preventing vanishing gradients. \end{description} \medskip \noindent**Symbols at a glance.**
\begin{center}\small \begin{tabular}{@{}ll@{}} \toprule **Symbol** & **Meaning**
\midrule $\mathbf{w}, b$ & Weights vector and scalar bias term
$\sigma(z)$ & Sigmoid activation function $\frac{1}{1+e^{-z}}$
$L(y, \hat{y})$ & Loss function measuring discrepancy between true label y and prediction $\hat{y}$
η & Learning rate parameter for gradient descent updates
K, P, S & Convolution kernel size, padding width, and stride step size
\bottomrule \end{tabular} \end{center}